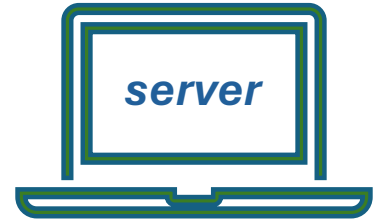


Shiny: Understanding the server

Ermis-Ioannis Michail-Delopoulos

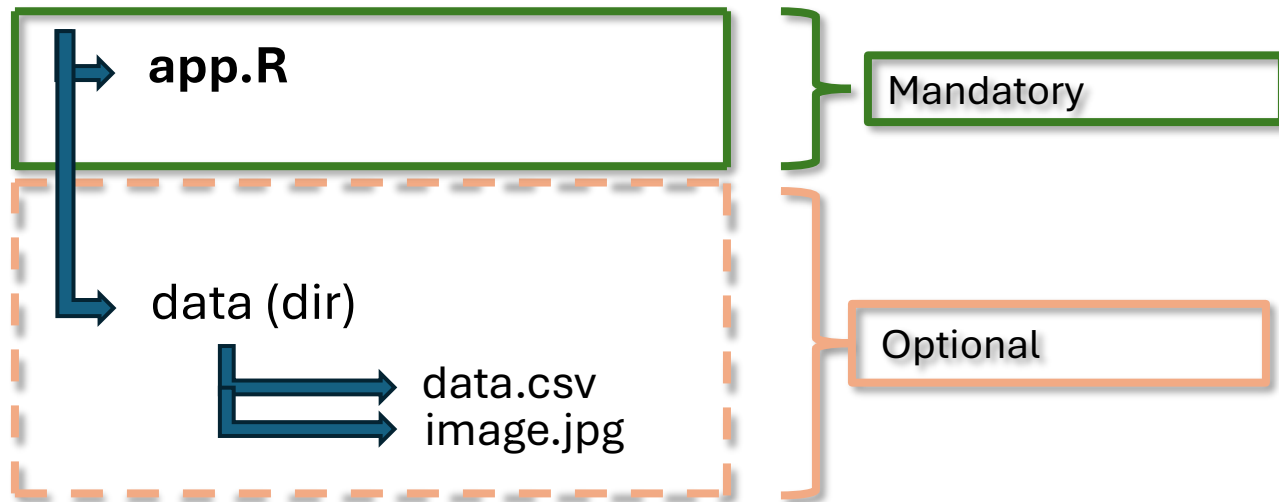
Learning Objectives



- **Understand the role of the server function in Shiny**
- **Learn how to link UI components (inputs & outputs) with server-side logic**
- **Grasp the concepts of reactivity and reactive context**

Quick Recap: Structure of Shiny App

Test_App (Directory)



The directory that you save app.R in becomes the working directory of your Shiny app

```
runApp("Test_App")
```

Quick Recap: App.R

- Comprises of 3 parts

- User Interface (Ui.R)
- Server function (server.R)
- Shiny app function



```
library(shiny)           # Imported Packages

# Global variables loaded here
df<-read.csv("data/data.csv")

ui <-
# Code for Ui logic

server <- function(input, output, session) {
# Code for Server logic
}

shinyApp(ui = ui, server = server)
```

Quick Recap : Input / Output components in Ui

```
ui <- fluidPage(  
  
  numericInput(inputId="count", label="Number of values",  
    value = 100),  
  
  textInput(inputId = "txt1", "First Name:"),  
  
  sliderInput(inputId = "slider1", min = 0, max = 10,  
    label = "Select value", value = 0),  
  
  actionButton(inputId = "assign", "Assign Button"),  
  
  textOutput(outputId = "textOutput1"),  
  plotOutput (outputId="plot1")  
  Etc..  
)
```

Number of values

100

First Name:

HELLO WORLD

Select value

0 10

Assign Button

[1] "HELLO WORLD"

What happens when we start a shiny app

1. Initialization (**Once**)

- Objects, packages, and data defined outside the server() function are loaded
- UI and the HTML blueprint are compiled

2. Server Setup (**Once** per user session)

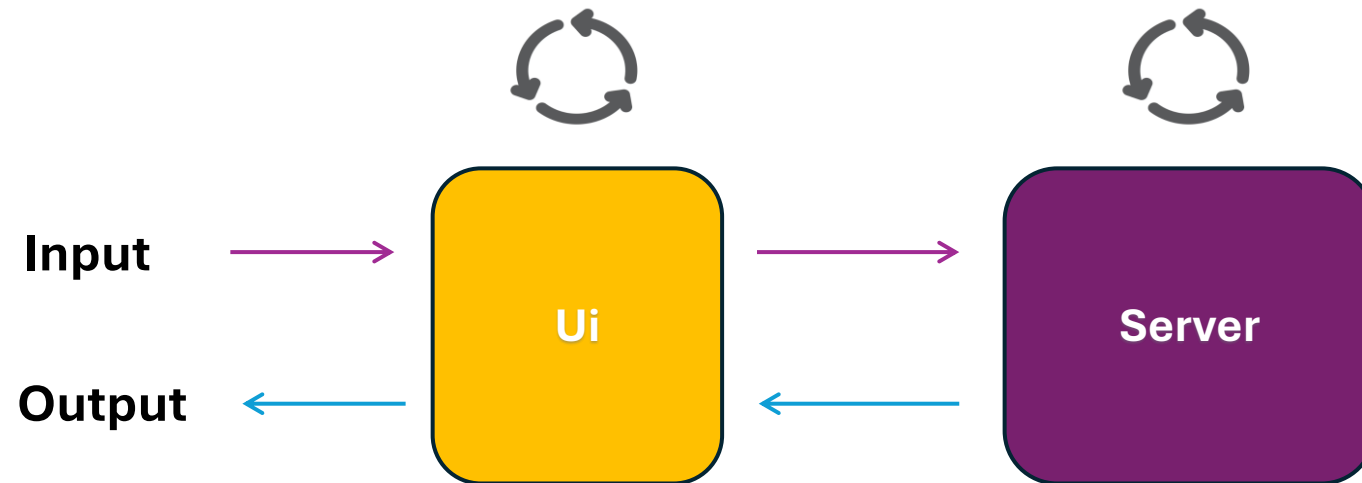
- Shiny creates a new environment and calls the server() function
- Each user session runs in its own isolated environment

3. Reactive Lifecycle (**Continuously** during the session)

- Tracks input-output dependencies
- Re-runs render*() blocks and updates outputs dynamically when inputs change

Shiny Reactivity

💡 The idea is that you don't need to tell shiny when to update the inputs and outputs. But Shiny does it automatically



The server function

Server() function has 3 parameters:

1. *input*
2. *output*
3. *session*

```
server <- function(input, output, session){...}
```

Handles advanced tasks (updates, URLs...)

*List-like, reactive, **READ-ONLY***

*List-like, reactive, **READ-ONLY***

Outputs in Server

Outputs are:

- List-like objects named according to the **outputId**, and they are used to sent the output to the output components in UI
- **Outputs** cannot be directly handled, they need a **reactive context**, which in this case is **render*() functions**.
- **Outputs** are **always** accompanied with **render*() functions**.

```
server <- function(input, output, session) {  
  output$textOutput1 <- renderText("Hello World!")  
}
```

```
ui <- fluidPage(  
  textOutput(outputId = "textOutput1")  
)
```



Render functions

- To access **outputs** and send them to **ui** we need to wrap them inside of **reactive context**

❑ Rendering functions

- `renderPlot ({ ... })`
- `renderTable ({ ... })`
- `renderText ({ ... })`,
- `etc.`



- Automatically tracks what inputs the output uses
- Converts the output of your R code into HTML suitable for display on a web page

Render functions

Types of output (in Ui)

Output function	Creates
<code>dataTableOutput</code>	DataTable
<code>htmlOutput</code>	raw HTML
<code>imageOutput</code>	image
<code>plotOutput</code>	plot
<code>tableOutput</code>	table
<code>textOutput</code>	text
<code>uiOutput</code>	raw HTML
<code>verbatimTextOutput</code>	text

Render* functions in server that send the reactive objects back to Ui

render function	creates
<code>renderDataTable</code>	DataTable
<code>renderImage</code>	images (saved as a link to a source file)
<code>renderPlot</code>	plots
<code>renderPrint</code>	any printed output
<code>renderTable</code>	data frame, matrix, other table like structures
<code>renderText</code>	character strings
<code>renderUI</code>	a Shiny tag object or HTML

Example Render function for images

```
server <- function(input, output, session) {  
  mydata <- read.csv("data/data.csv")  
  
  output$table1 <- renderDataTable ({ mydata })  
}
```

```
ui <- fluidPage(  
  dataTableOutput("table1")  
)
```

Show 10 entries

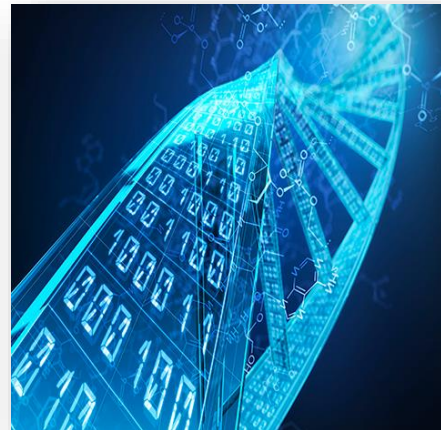
Search: _____

	species	island	bill_length_mm	bill_depth_mm	flip
1	Adelie	Torgersen	39.1	18.7	
2	Adelie	Torgersen	39.5	17.4	
3	Adelie	Torgersen	40.3	18	
4	Adelie	Torgersen			

Example Render function for images

```
server <- function(input, output, session) {  
  output$image <- renderImage({  
    list(src = "data/image.jpg"  
         height = "100%")  
  })  
}
```

```
ui <- fluidPage(  
  imageOutput("image")  
)
```



Inputs in Server (1)

Inside the **server**, we can access any input components value with the accessor `input$` and the **inputId**:

`input`

<code>\$count</code>	→ numeric (e.g., 42)
<code>\$txt1</code>	→ character (e.g., "Alice")
<code>\$slider1</code>	→ numeric (e.g., 3)

- `numericInput(inputId="count", label="Number of values", value = 100),`

→ `input$count`

- `textInput(inputId = "txt1", "First Name:"),`

→ `input$txt1`

Inputs in Server (2)

BUT, unlike a typical list, input objects in server are:

- **Read-only** (cannot be directly manipulated)
- To handle them you need **reactive context** (more later on)

```
ui <- fluidPage(  
  
  textInput(inputId = "txt1", "First Name:"),  
  textOutput(outputId = "textOutput1")  
)
```

```
server <- function(input, output,  
session) {
```

```
X<-input$txt1 ✗  
input$txt1<-"hello" ✗
```

```
Output$textOutput1<-renderPrint({  
  X<-input$txt1  
  X  
})  
  
}
```

Reactive expressions

- To access **inputs** inside the server except render functions we can also wrap them inside of another type of **reactive context**, the reactive expressions

❑ Reactive expressions

- `reactive({ ... })`
- `eventReactive(event,{ ... })`



- Can access reactive inputs or other reactive expressions
- Automatically track a user input
- Cache the results & Return a value

Reactive

```
ui <- fluidPage(  
  
  textInput(inputId = "txt1", "First Name:"),  
  textOutput(outputId = "textOutput1"),  
  textOutput(outputId = "textOutput2")  
)
```

```
server <- function(input, output,  
session) {
```

```
  x<-reactive({ input$txt1 })
```

```
  output$textOutput1 <- renderPrint({  
    Print ( x() )  
  })
```

```
  output$textOutput2 <- renderPrint({  
    Print ( x() )  
  })
```

```
}
```

First Name:

[1] "Hello World"

[1] "Hello World"

After creating a reactive
variable to call it again
we need parenthesis,
**So, to call x we call it
as x()**

EventReactive

- Event reactives are like reactives, but they are used together with assign buttons or for timed validation
- They have two arguments inside, the first is the dependency and then it's the regular reactive argument

```
ui <- fluidPage(  
  
  textInput(inputId = "txt1", "First Name:"),  
  textOutput(outputId = "textOutput1")  
  actionButton(inputId = "assign", "Assign Button"),  
)
```

```
server <- function(input, output,  
  session) {  
  
  input_value <- eventReactive(input$assign, {  
    input$txt1 })  
  
  output$textOutput1 <- renderPrint({  
    paste("Your assigned input", input_value())  
  })  
  
}
```

Number of values

100

First Name:

Hello World

Select value



Assign Button

Your assigned input Hello World

Observers

- **Observers** are a type of reactive context for observing something happening.
- They differ from **reactive expressions** and **rendering functions**, as **observers** don't return a value

❑ Observers

- `observe({ ... })`
- `observeEvent(event, { ... })`



- can access reactive inputs and reactive expressions, but they don't return a value
- they are used for their side effects

Observers

```
server <- function(input, output, session) {  
  
  observe ({showNotification(input$txt1 ,type =  
    "message")  
  })  
  
}
```

```
server <- function(input, output, session) {  
  
  observeEvent(input$assign, {  
    showNotification("Button clicked!", type =  
      "message")  
  })  
  
}
```

Notifications

- In Shiny, notifications are created with **showNotification()**, and stack in the bottom right of the page.

```
showNotification(  
  "message"  
  duration = 5,  
  closeButton = TRUE,  
  type = c("default", "message", "warning", "error")  
)
```

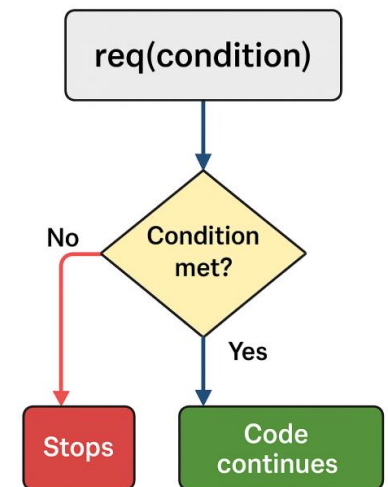
Using req() in Shiny

💡 What is req()?

- A validation function that ensures required conditions or inputs are available before proceeding
- If the condition is not met, Shiny stops the reactive or render function where req() is called

✅ When to use it?

- To prevent errors when inputs are empty or NULL
- To stop downstream code until a user selects or provides input



Using req() in Shiny

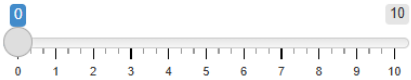
```
server <- function(input, output, session) {  
  input_value1 <- reactive({  
    req(input$txt1)  
  })  
  
  input_value2 <- reactive({  
    req(nchar(input$txt2) > 2)  
  })  
  
  output$textOutput1 <- renderText({  
    paste("Your Name:", input_value1()),  
    "Your Last Name:", input_value2())  
  })  
}
```

Number of values

First Name:

Last Name:

Select value

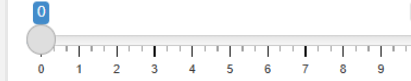


Number of values

First Name:

Last Name:

Select value



Your Name Hello Your Last Name World

Hands-On: Complete the Shiny Server

Grab the starter code

- Clone / download from Github:
[code_exercises](#),
L3_Exercise.

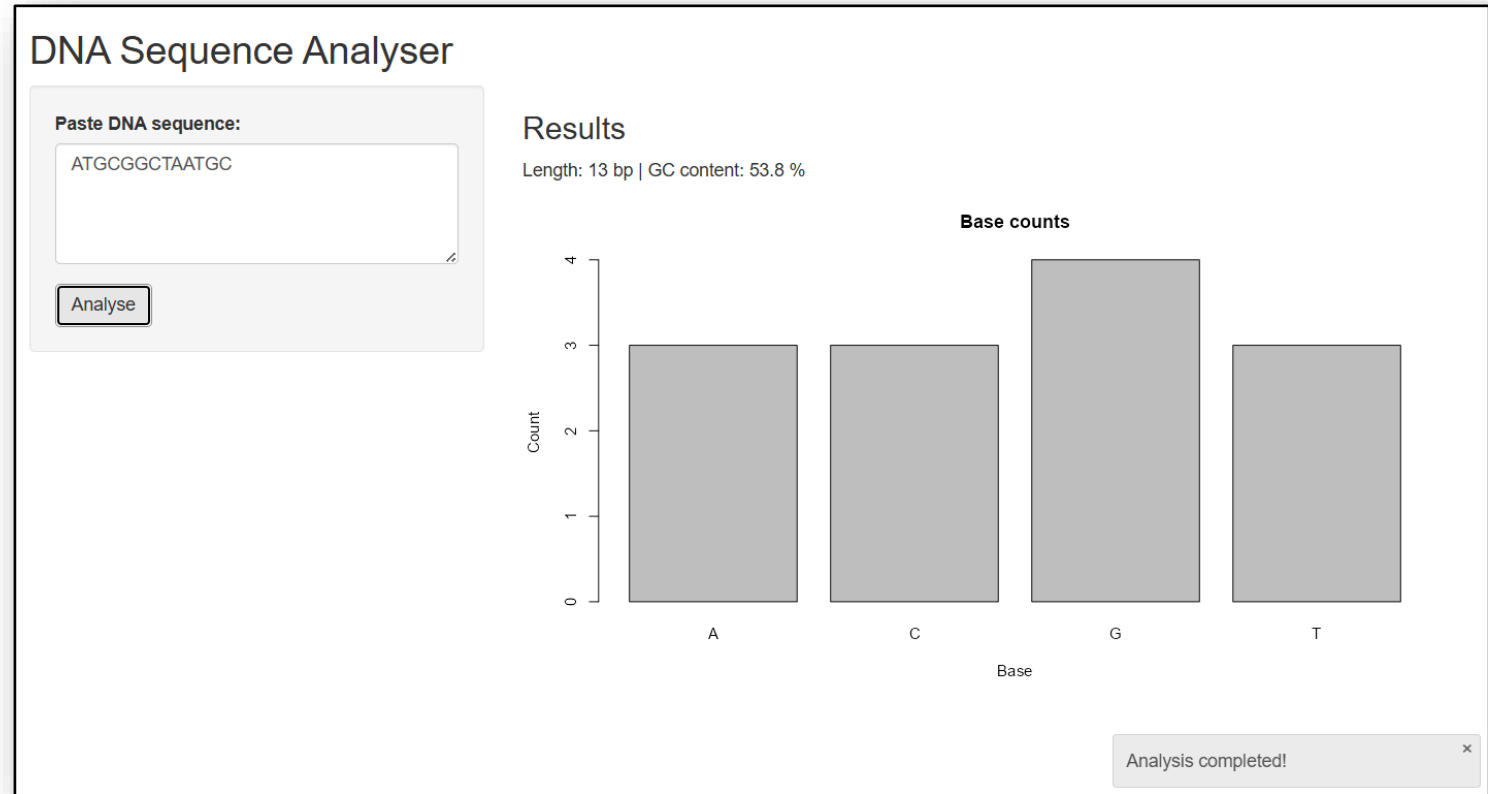
Open App.R

- The UI is finished: your mission is to **fill** in the numbered **TODOs** in the **server** block.

Build & test

- Implement TODO, click Run App, confirm It works

Approximate time: 15–20 minutes.



How app should look in the end

The reactive graph

Reactive graph helps us to understand the order of execution, which describes how inputs and outputs are connected

Three components of reactive execution in Shiny are:

1. reactive inputs



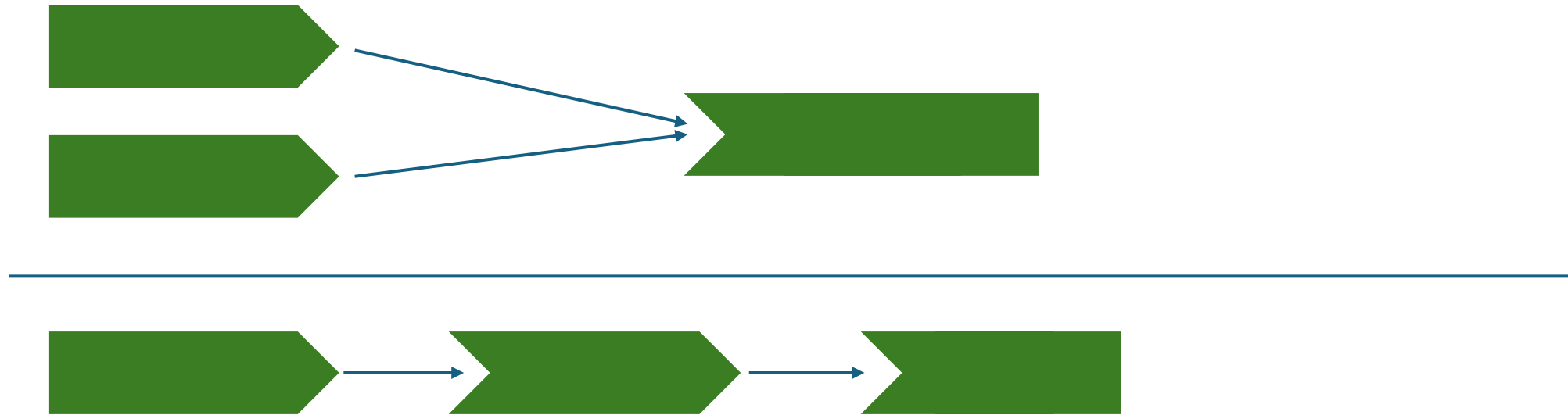
2. reactive expressions



3. reactive outputs

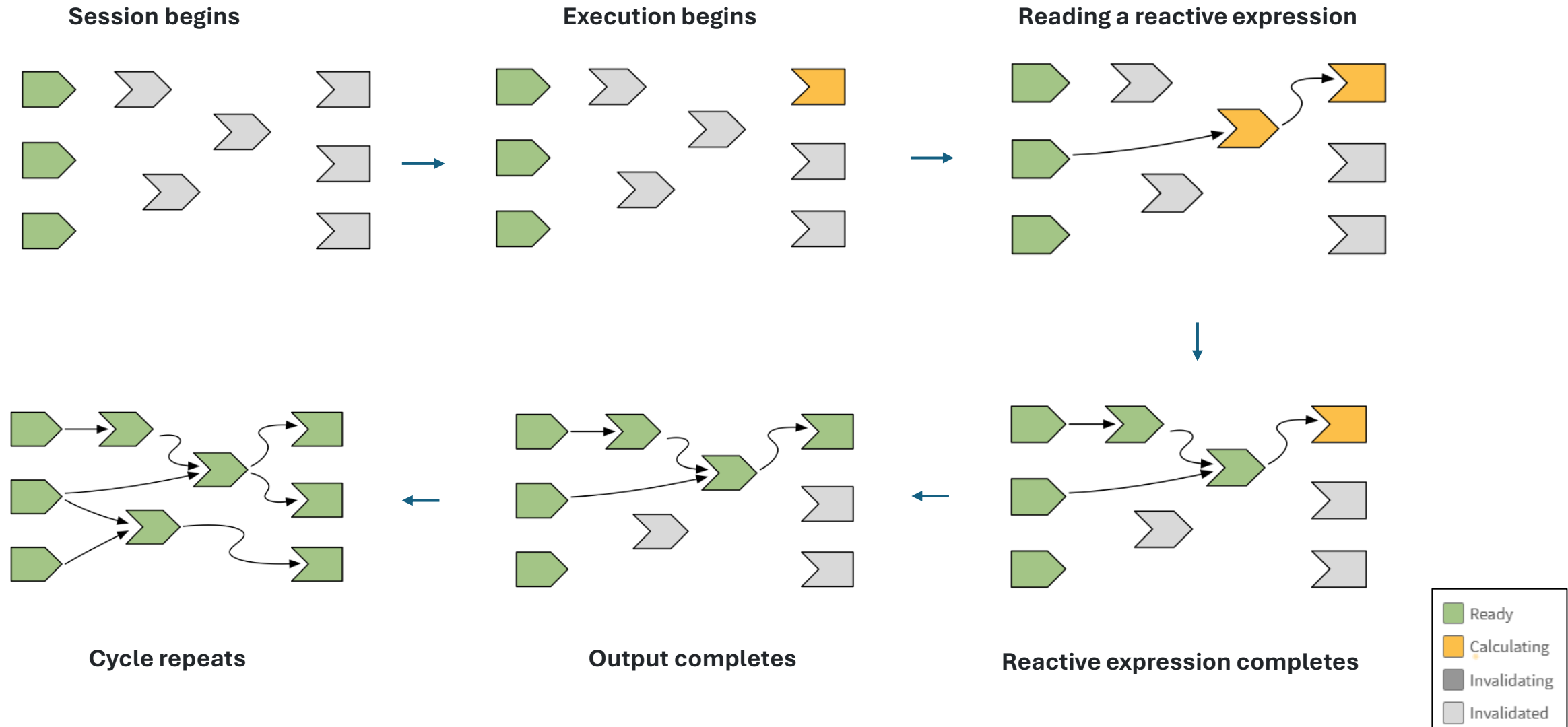


The reactive graph

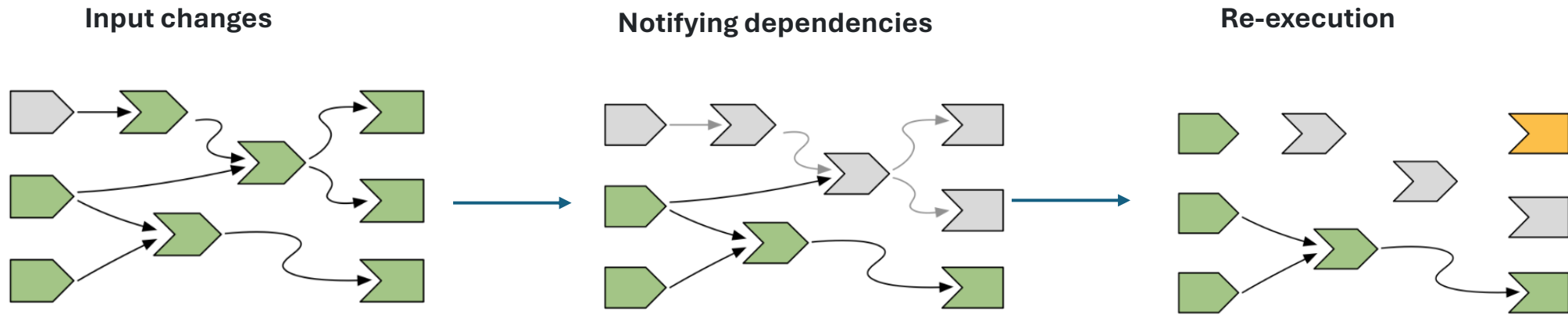


The connections between the components are directional, with the arrows indicating the direction of reactivity

The reactive graph



The reactive graph



The reactive graph - reactlog package

Using **reactlog** package

```
# Install reactlog package :
```

```
>> install.packages("reactlog")
```

```
# Add reactlog library
```

```
>> library(shiny)
```

```
>> library(reactlog)
```

```
# tell shiny to log all reactivity
```

```
>> reactlog_enable()
```

```
# run a shiny app
```

```
>> shiny::runApp(app) #app (directory where app.R is)
```

```
# once app has closed, display reactlog from shiny
```

```
>>shiny::reactlogShow()
```

Or while Shiny app is running, press Ctrl+F3 (Mac: Cmd+F3) to launch the reactlog application

The reactive graph – reactlog package

